

Pixel Battle — Combat-Feel Polish (Sub-project A)

Implementation Plan

Pixel Battle — Combat-Feel Polish (Sub-project A)

Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Make combat *feel* tighter and weightier — smaller framing, no out-of-range whiff animations, and a strong hitstop + recoil + screen-shake reaction on hits — without retraining the RL policy.

Architecture: Four focused changes. Engine-layer hitstop (`battle.py` freeze counter), a per-skill attack-range gate (`env.py`), a camera zoom-out + crit screen-shake (`play.py`), and a bigger hit-reaction pose (`poses.py`). All changes leave the RL observation, action space, and reward untouched — the trained checkpoint stays valid, no retrain.

Tech Stack: Python 3.10, pygame (headless SDL dummy), pytest, ffmpeg, stable-baselines3 (only to load the existing checkpoint for the validation render).

Design spec: `docs/superpowers/specs/2026-05-22-pixel-battle-combat-feel-design.md`

File Structure

File	Status	Responsibility
<code>pixel_battle/engine/battle.py</code>	modify	HITSTOP_MS / HITSTOP_MS_HEAVY constants, <code>_hitstop_for</code> helper, <code>_hitstop_remaining</code> counter; <code>tick_ms</code> freezes while it is positive; <code>_resolve_attack_hit</code> sets it on a hit
<code>pixel_battle/rl/env.py</code>	modify	<code>_apply_action</code> — replace the single <code>ATTACK_GATE_RANGE</code> gate with a per-skill-type gate (melee actions at <code>MELEE_RANGE</code> , special/cd at <code>SPECIAL_RANGE</code>)

File	Status	Responsibility
<code>pixel_battle/rl/play.py</code>	modify	<code>CAM_ZOOM</code> 1.7 → 1.45; <code>camera_shake_offset</code> helper + crit-triggered screen shake in <code>_render_fight</code>
<code>pixel_battle/rl/poses.py</code>	modify	enlarge <code>HIT_POSE</code> (more dramatic recoil)
<code>pixel_battle/tests/test_poses.py</code>	modify	update the stale <code>CAM_VIEW_H</code> comment in the visual-safety test
<code>pixel_battle/tests/test_hitstop.py</code>	new	hitstop freeze + hit-sets-hitstop tests
<code>pixel_battle/tests/test_env_attack_gate.py</code>	new	per-skill attack-gate tests
<code>pixel_battle/tests/test_camera_shake.py</code>	new	<code>camera_shake_offset</code> tests

No RL retrain: hitstop inserts inert frozen ticks; the attack gate only changes which attack actions no-op. Neither touches the observation, action space, or reward.

Task 1: Engine-layer hitstop

Files: - Modify: `pixel_battle/engine/battle.py` - Test: `pixel_battle/tests/test_hitstop.py` (new)

`battle.py` current facts (verified): module constants are at lines ~21-52 (`STAGGER_MS = 300` at line 24). `Battle`'s constructor sets `self.elapsed_ms = 0` at line 89. The per-tick entry point is `tick_ms(self, dt_ms, skip_ai=False)` at line 101, whose first line is `self.elapsed_ms += dt_ms` (line 102). `_resolve_attack_hit` (line 248) determines `is_crit` at line 274 and ends the hit-resolved block around `attacker.last_attack_ms = self.elapsed_ms` (line 305).

☐ Step 1: Write the failing tests

```
# pixel_battle/tests/test_hitstop.py
"""Engine-layer hitstop – a short freeze on every hit."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.engine.battle import HITSTOP_MS, HITSTOP_MS_HEAVY, _hitstop_for
from pixel_battle.rl.env import PixelBattleEnv

def test_hitstop_for_crit_is_heavier():
    assert _hitstop_for(False) == HITSTOP_MS
    assert _hitstop_for(True) == HITSTOP_MS_HEAVY
```

```

assert HITSTOP_MS_HEAVY > HITSTOP_MS > 0

def test_tick_freezes_and_does_not_advance_clock():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    b._hitstop_remaining = HITSTOP_MS           # force a freeze
    before = b.elapsed_ms
    b.tick_ms(16)
    assert b.elapsed_ms == before                # clock paused during hitstop
    assert b._hitstop_remaining == HITSTOP_MS - 16

def test_tick_resumes_after_hitstop_drains():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    b._hitstop_remaining = 10
    b.tick_ms(16)                                # drains 10 -> -6, still a frozen call
    assert b._hitstop_remaining <= 0
    before = b.elapsed_ms
    b.tick_ms(16)                                # no longer frozen -> clock advances
    assert b.elapsed_ms > before

def test_a_landed_hit_sets_hitstop():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    atk, dfn = env.left, env.right
    atk.pos_x, dfn.pos_x = 240.0, 300.0          # 60 px apart - inside melee range
    atk.accuracy = 1.0                          # guarantee the accuracy roll passes
    b._hitstop_remaining = 0
    b._resolve_attack_hit(atk, dfn)
    assert b._hitstop_remaining >= HITSTOP_MS    # HITSTOP_MS, or _HEAVY on a crit

```

☐ Step 2: Run the tests to verify they fail

Run: `python -m pytest pixel_battle/tests/test_hitstop.py -v` Expected: FAIL — `ImportError: cannot import name 'HITSTOP_MS'.`

☐ Step 3: Add the constants + helper to `battle.py`

In the module constants block (next to `STAGGER_MS = 300`, line ~24), add:

```
HITSTOP_MS = 50          # freeze on a normal hit – the "impact" pause
HITSTOP_MS_HEAVY = 100   # freeze on a crit – heavier hits read weightier
```

After the module constants block (above `class EventType`), add the helper:

```
def _hitstop_for(is_crit: bool) -> int:
    """Freeze duration (ms) for a hit – longer for a crit."""
    return HITSTOP_MS_HEAVY if is_crit else HITSTOP_MS
```

☐ Step 4: Initialise the counter + freeze `tick_ms`

In `Battle`'s constructor, directly after `self.elapsed_ms = 0` (line 89), add:

```
self._hitstop_remaining = 0
```

At the very top of `tick_ms` (before `self.elapsed_ms += dt_ms`, line 102), add the freeze gate:

```
def tick_ms(self, dt_ms: int, skip_ai: bool = False) -> None:
    if self._hitstop_remaining > 0:
        self._hitstop_remaining -= dt_ms
        return
    self.elapsed_ms += dt_ms
    # ... rest of tick_ms unchanged ...
```

☐ Step 5: Set hitstop when a hit lands

In `_resolve_attack_hit`, directly after `attacker.last_attack_ms = self.elapsed_ms` (line 305), add:

```
self._hitstop_remaining = _hitstop_for(is_crit)
```

`is_crit` is already in scope (assigned at line 274).

☐ Step 6: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_hitstop.py -v` Expected: PASS (4/4).

☐ Step 7: Run the engine regression tests

Run: `python -m pytest pixel_battle/tests/test_battle_no_lock.py`

`pixel_battle/tests/test_env_symmetry_and_kick.py pixel_battle/tests/test_lol_champions.py -v` Expected: all green (hitstop must not break existing battle behaviour).

☐ Step 8: Commit

```
git add pixel_battle/engine/battle.py pixel_battle/tests/test_hitstop.py
git commit -m "feat(pixel-battle/engine): engine-layer hitstop on hits"
```

Task 2: Per-skill attack-range gate

Files: - Modify: `pixel_battle/rl/env.py` - Test: `pixel_battle/tests/test_env_attack_gate.py` (new)

`env.py` current facts (verified): `ATTACK_GATE_RANGE = 145` at line 35. `_apply_action(self, me, opp, action)` at line 174 computes `in_attack_range = abs(me.pos_x - opp.pos_x) <= ATTACK_GATE_RANGE` (line 185) and gates actions 4/5/7/8 with it. `MELEE_RANGE` (110) and `SPECIAL_RANGE` (130) live in `pixel_battle/engine/physics.py` (`battle.py` imports them from there — match that import path).

☐ Step 1: Write the failing tests

```
# pixel_battle/tests/test_env_attack_gate.py
"""Per-skill attack-range gate – attacks no-op when out of the skill's range."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.engine.physics import MELEE_RANGE, SPECIAL_RANGE

def _env_at(distance):
    env = PixelBattleEnv(seed=1)
    env.left.pos_x = 240.0
    env.right.pos_x = 240.0 + distance
    env.left.mp = 80 # enough MP for any special
    return env

_MID_BAND = (MELEE_RANGE + SPECIAL_RANGE) // 2 # ~120: past melee, within special

def test_basic_attack_gated_out_past_melee_range():
    env = _env_at(_MID_BAND)
    env._apply_action(env.left, env.right, 4) # basic
    assert env.left.action_state != "attacking"
```

```

def test_basic_attack_fires_within_melee_range():
    env = _env_at(MELEE_RANGE - 30)
    env._apply_action(env.left, env.right, 4)      # basic
    assert env.left.action_state == "attacking"

def test_special_fires_in_the_mid_band_where_basic_was_gated():
    env = _env_at(_MID_BAND)
    env._apply_action(env.left, env.right, 7)      # special
    assert env.left.action_state == "attacking"

def test_special_gated_out_past_special_range():
    env = _env_at(SPECIAL_RANGE + 30)
    env._apply_action(env.left, env.right, 7)      # special
    assert env.left.action_state != "attacking"

```

☐ Step 2: Run the tests to verify they fail

Run: `python -m pytest pixel_battle/tests/test_env_attack_gate.py -v` Expected: FAIL — at the mid-band distance the current single 145 px gate lets the basic attack fire, so `test_basic_attack_gated_out_past_melee_range` fails.

☐ Step 3: Import the ranges and replace the gate

At the top of `pixel_battle/rl/env.py`, add to the imports (match the path `battle.py` uses for these):

```

from pixel_battle.engine.physics import MELEE_RANGE, SPECIAL_RANGE

```

Delete the now-unused constant `ATTACK_GATE_RANGE = 145` (line 35).

Replace the body of `_apply_action` (lines 174-205) with the per-skill-gated version:

```

def _apply_action(self, me: Character, opp: Character, action: int):
    if me.action_state in ("attacking", "hit_stagger", "ko"):
        return
    # Direction toward opponent (+1 if opp to my right, -1 if to my left)
    fwd = 1 if opp.pos_x > me.pos_x else -1
    # Per-skill attack gate: an attack issued out of the skill's reach is
    # a no-op (no doomed whiff animation). Melee actions (basic, kick)
    # gate at MELEE_RANGE; special and cd actions at SPECIAL_RANGE.
    # (Ultimate is ungated – it always connects.)
    dist = abs(me.pos_x - opp.pos_x)

```

```

if action == 1:                                # back (away from opp)
    me.vel_x = -3.0 * fwd
    me.facing = fwd                            # still face opp while backpedaling
elif action == 2:                              # forward (toward opp)
    me.vel_x = 3.0 * fwd
    me.facing = fwd
elif action == 3 and me.on_ground:             # jump
    me.vel_y = -8.0
    me.on_ground = False
elif action == 4 and dist <= MELEE_RANGE:      # basic attack
    self.battle._start_attack_with_kind(me, opp, "basic")
elif action == 5 and dist <= SPECIAL_RANGE:    # cd skill
    self.battle._start_attack_with_kind(me, opp, "cooldown")
elif action == 6 and me.ultimate_ready():
    # cinematic_pause=False - no multi-second freeze in the RL render
    self.battle._trigger_ultimate(me, opp, cinematic_pause=False)
elif action == 7 and dist <= SPECIAL_RANGE:    # special skill
    self.battle._start_attack_with_kind(me, opp, "special")
elif action == 8 and dist <= MELEE_RANGE:      # kick
    self.battle._start_attack_with_kind(me, opp, "kick")

```

☐ Step 4: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_env_attack_gate.py -v` Expected: PASS (4/4).

☐ Step 5: Run the RL regression tests

Run: `python -m pytest pixel_battle/tests/test_env_symmetry_and_kick.py`

`pixel_battle/tests/test_play_multi_imports.py -v` Expected: all green. If any test referenced `ATTACK_GATE_RANGE` by name, update it to the new per-skill logic (it must verify real gating behaviour, not the deleted constant).

☐ Step 6: Commit

```

git add pixel_battle/rl/env.py pixel_battle/tests/test_env_attack_gate.py
git commit -m "feat(pixel-battle/rl): per-skill attack-range gate - no out-of-range whiffs"

```

Task 3: Smaller characters (camera zoom-out)

Files: - Modify: `pixel_battle/rl/play.py` - Modify: `pixel_battle/tests/test_poses.py` (stale comment only)

`play.py` current facts (verified): `CAM_ZOOM = 1.7` at line 46; `CAM_VIEW_W = int(WIDTH / CAM_ZOOM)` and `CAM_VIEW_H = int(HEIGHT / CAM_ZOOM)` derive from it.

☐ Step 1: Lower `CAM_ZOOM`

In `pixel_battle/rl/play.py`, change `CAM_ZOOM` from `1.7` to `1.45`:

```
CAM_ZOOM = 1.45           # was 1.7 – slight zoom-out: smaller fighters, more stage
```

`CAM_VIEW_W / CAM_VIEW_H` recompute automatically (`854 / 1.45 ≈ 589` tall).

☐ Step 2: Update the stale comment in the visual-safety test

In `pixel_battle/tests/test_poses.py`, the visual-safety test has a comment derived from the old zoom (`CAM_VIEW_H = 502`, `~411 px` headroom). With `CAM_ZOOM = 1.45` the camera view is `~589 px` tall, so `~483 px` above the feet are visible. Update the comment so it stays accurate (the `_MAX_HEIGHT = 400` / `_MAX_HALF_W = 260` constants stay — they are still conservative and correct):

```
# play.py camera shows CAM_VIEW_H ~589 world px (CAM_ZOOM 1.45) with the
# floor framed ~82% down, so ~483 px above the feet are on-screen.
_MAX_HALF_W = 260      # gross-error guard on horizontal splay from pos_x
_MAX_HEIGHT = 400      # figure + weapon must stay under the camera's top edge
```

☐ Step 3: Run the full suite for regressions

Run: `python -m pytest pixel_battle/tests/ -q` Expected: all green — a lower zoom gives the camera *more* headroom, so the visual-safety test and any camera/HUD test still pass.

☐ Step 4: Commit

```
git add pixel_battle/rl/play.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): zoom camera out – smaller, better-framed fighters"
```

Task 4: Bigger hit-reaction pose

Files: - Modify: `pixel_battle/rl/poses.py`

`poses.py` current fact (verified): `HIT_POSE = FigurePose(torso_lean=-22.0, front_arm=(250.0, -60.0), back_arm=(290.0, -60.0), front_leg=(110.0, 50.0), back_leg=(70.0, 40.0), weapon_deg=300.0)`. The visual-safety test `test_all_poses_keep_feet_planted_and_in_frame` already exercises the `hit` pose — it is the regression lock for this change.

☐ Step 1: Enlarge HIT_POSE

In `pixel_battle/rl/poses.py`, replace the `HIT_POSE` definition with a more dramatic recoil — torso thrown further back, arms flailing wider and more bent, knees buckling harder:

```
HIT_POSE = FigurePose(
    torso_lean=-38.0,
    front_arm=(235.0, -80.0), back_arm=(305.0, -80.0),
    front_leg=(120.0, 70.0), back_leg=(62.0, 58.0),
    weapon_deg=295.0)
```

(Starting values — tuned in Task 6’s validation render.)

☐ Step 2: Run the pose tests — the visual-safety lock must still pass

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: PASS — including `test_all_poses_keep_feet_planted_and_in_frame`, which proves the enlarged `HIT_POSE` keeps its feet planted and stays inside the camera frame. If it fails as “too tall”/“too wide”, pull the most extreme angle back toward neutral until it passes (do not loosen the test constants).

☐ Step 3: Run the full suite

Run: `python -m pytest pixel_battle/tests/ -q` Expected: all green.

☐ Step 4: Commit

```
git add pixel_battle/rl/poses.py
git commit -m "feat(pixel-battle/rl): bigger hit-reaction recoil pose"
```

Task 5: Screen shake on crits

Files: - Modify: `pixel_battle/rl/play.py` - Test: `pixel_battle/tests/test_camera_shake.py` (new)

`play.py` current fact (verified): the camera block in `_render_fight` is:

```
view_x = int(cam_x - CAM_VIEW_W / 2)
view_x = max(0, min(WIDTH - CAM_VIEW_W, view_x))
view_y = max(0, min(HEIGHT - CAM_VIEW_H, CAM_VIEW_Y))
sub = world.subsurface((view_x, view_y, CAM_VIEW_W, CAM_VIEW_H))
```

`_render_fight` already has a per-frame loop over new battle events (`for ev in env.battle.events[prev_ev_n:]` , with the type read as `et = ev.type.value`).

☐ Step 1: Write the failing tests

```
# pixel_battle/tests/test_camera_shake.py
"""Crit screen-shake camera offset."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.play import camera_shake_offset, SHAKE_FRAMES, SHAKE_MAG

def test_no_offset_when_not_shaking():
    assert camera_shake_offset(0) == (0, 0)
    assert camera_shake_offset(-2) == (0, 0)

def test_offset_within_magnitude_while_shaking():
    for frames in range(1, SHAKE_FRAMES + 1):
        for _ in range(40):
            dx, dy = camera_shake_offset(frames)
            assert abs(dx) <= SHAKE_MAG
            assert abs(dy) <= SHAKE_MAG

def test_offset_decays_toward_end_of_shake():
    # The peak possible magnitude at frame 1 is smaller than at full strength.
    early_peak = SHAKE_MAG * (SHAKE_FRAMES / SHAKE_FRAMES)
    late_peak = SHAKE_MAG * (1 / SHAKE_FRAMES)
    assert late_peak < early_peak
```

☐ Step 2: Run the tests to verify they fail

Run: `python -m pytest pixel_battle/tests/test_camera_shake.py -v` Expected: FAIL —
ImportError: cannot import name 'camera_shake_offset'.

☐ Step 3: Add the shake helper to `play.py`

Near the camera constants in `play.py` (by `CAM_ZOOM`), add the constants and helper (`random` is a stdlib import — add `import random` at the top if not already present):

```
SHAKE_FRAMES = 8          # frames a crit screen-shake lasts
SHAKE_MAG = 14            # peak shake offset, world px
```

```
def camera_shake_offset(frames_remaining: int) -> tuple:
    """Decaying random camera offset for a crit screen-shake.

    Returns (0, 0) when not shaking; otherwise a random (dx, dy) whose
    magnitude decays linearly as `frames_remaining` counts down.
    """
    if frames_remaining <= 0:
        return (0, 0)
    mag = int(SHAKE_MAG * (frames_remaining / SHAKE_FRAMES))
    return (random.randint(-mag, mag), random.randint(-mag, mag))
```

☐ Step 4: Wire the shake into `_render_fight`

In `_render_fight`:

1. Before the per-frame render loop, initialise the shake counter:

```
shake_frames = 0
```

2. In the per-frame event loop (where `et = ev.type.value` is read), add a branch — a crit triggers the shake:

```
if et == "crit":
    shake_frames = SHAKE_FRAMES
```

3. Replace the camera block with a shake-offset version (the `max/min` clamps keep the crop inside the world surface, so the shake can never read off-surface):

```
sx, sy = camera_shake_offset(shake_frames)
view_x = int(cam_x - CAM_VIEW_W / 2) + sx
view_x = max(0, min(WIDTH - CAM_VIEW_W, view_x))
view_y = max(0, min(HEIGHT - CAM_VIEW_H, CAM_VIEW_Y + sy))
sub = world.subsurface((view_x, view_y, CAM_VIEW_W, CAM_VIEW_H))
if shake_frames > 0:
    shake_frames -= 1
```

☐ Step 5: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_camera_shake.py -v` Expected: PASS (3/3).

☐ Step 6: Run the `play.py` regression tests

Run: `python -m pytest pixel_battle/tests/test_play_richness.py`

`pixel_battle/tests/test_skill_vfx.py -v` Expected: all green (the `_render_fight` change must not break the HUD/effects helpers).

☐ Step 7: Commit

```
git add pixel_battle/rl/play.py pixel_battle/tests/test_camera_shake.py
git commit -m "feat(pixel-battle/rl): crit screen-shake"
```

Task 6: Full test sweep + validation render

Files: none (verification only; possible tuning commits)

☐ Step 1: Run the entire test suite

Run: `python -m pytest pixel_battle/tests/ -q` Expected: all green. Fix any regression before continuing.

☐ Step 2: Render a validation episode

Run: `python -m pixel_battle.rl.play` Expected: `pixel_battle/output/rl_play/episode.mp4` is regenerated with no Python error. The match must still complete normally — confirm hitstop did not cause an early truncation (the episode should end on a KO, similar length to before plus ~1-2 s of accumulated hitstop).

☐ Step 3: Inspect the output

Run: `ffprobe -v error -show_entries format=duration,size -of default=noprint_wrappers=1 pixel_battle/output/rl_play/episode.mp4`

Watch the mp4 and check against the spec's success criteria: characters are smaller and better framed; hits land with a clear hitstop pause + bigger recoil + (on crits) a screen shake; attacks are no longer thrown at out-of-range opponents.

• ☐ Step 4: Tune if needed

- Characters still too large / too small → adjust `CAM_ZOOM` in `play.py`.
- Hitstop too short / too long → adjust `HITSTOP_MS` / `HITSTOP_MS_HEAVY` in `battle.py`.
- Recoil not punchy enough / broken framing → adjust `HIT_POSE` angles in `poses.py`.
- Shake too weak / too strong → adjust `SHAKE_MAG` / `SHAKE_FRAMES` in `play.py`.

- Fights look passive (agent hovers out of range) → widen the gate slightly (gate at `MELEE_RANGE + margin` / `SPECIAL_RANGE + margin`); if that does not help, it is a signal for Sub-project B's retrain.

After any tuning change, re-run `python -m pytest pixel_battle/tests/ -q` and re-render.

☐ Step 5: Commit any tuning changes

```
git add -A
git commit -m "tune(pixel-battle): combat-feel validation tuning"
```

Self-Review

Spec coverage — every spec section maps to a task: - §5.1 smaller characters → Task 3 - §5.2 per-skill attack-range gate → Task 2 - §5.3 engine-layer hitstop → Task 1 - §5.4 bigger hit reaction: enlarged `HIT_POSE` → Task 4; crit screen shake → Task 5 - §6 no retrain → no task touches the observation / action space / reward (verified per task) - §8 error handling → `_hitstop_remaining` only decremented while `> 0` (Task 1); shake offset `(0, 0)` when not shaking (Task 5) - §9 testing → Tasks 1, 2, 5 are TDD; Task 4 is locked by the existing visual-safety test; Task 6 runs the full sweep - §10 validation → Task 6

Deviation from spec §5.3/§5.4: the spec said heavy hitstop and screen shake on “a crit or ultimate”. This plan applies them on **crits** only — ultimates already have their own multi-second cinematic treatment (`ULTIMATE_DURATION_MS`), resolve through a separate code path (`_trigger_ultimate`, not `_resolve_attack_hit`), and do not need hitstop. Reaching into the ultimate path is unnecessary scope (YAGNI). Normal/crit hits — the rapid exchanges the user wants punctuated — are fully covered.

No placeholders — all constants are concrete starting values; Task 6 is the explicit tuning pass.

Type consistency — checked: `HITSTOP_MS` / `HITSTOP_MS_HEAVY` / `_hitstop_for` / `_hitstop_remaining` (Task 1); `MELEE_RANGE` / `SPECIAL_RANGE` imports and the `dist` gate (Task 2); `CAM_ZOOM` (Task 3); `HIT_POSE` `FigurePose` fields match the existing dataclass (Task 4); `camera_shake_offset` / `SHAKE_FRAMES` / `SHAKE_MAG` (Task 5).